

701968234901768023405879312016
670980321768767

WVPOYFXI LFJXI G I E
RGT MPHLOM ZOI P
BXINZWHYGI JLG L
OLI EPC OQUTJPL QDZC
NYDWIGL JYI X

Week 6

2. UNIX Process Control (III)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Wednesday, October 8th, 2025



774862586034187069210783468019
701968234901768023405879312016
670980321768767

DEFYXHQGRUOSGRP RN	3236343972474116
QFJWQJXVRI DKUEVP	893833235594578
KGWBGQFQHP RSUP RU	0310019701552750
XLRWILKZDCSYUUM I	78231501010516
HMSPUYN	78231501010516

0011001110011
11111010000100
110000000111
00111100111100
00101111111100
0000011101101
11001100111100



University of Windsor

Content

- 1 Differentiating a process: `exec()`
- 2 Changing directories: `chdir()`
- 3 System scheduling priority
- 4 Process groups

Part 0

DataCamp



University of Windsor

DataCamp

- About DataCamp
(www.DataCamp.com)
- Register ASAP:
https://www.datacamp.com/groups/shared_links/f3456d5152da452676dd2df4621761023f69268646bd78112c83b978a1fb4cbc
- Do your DataCamp Assignment: *An Introduction to bash Scripting*
- Issue and save the certificate.



Part I

Differentiating a Process: `exec ( )`



The `exec()` family of system calls allows a process to replace its current code, data and stack with those of another program.

Synopsis

```
int execl(const char *pathname, [const char  
*argi, ]* NULL)  
int execlp(const char *pathname, [const  
char *argi, ]* NULL)  
int execv(const char *filename, const char  
*argv[])  
int execvp(const char *filename, const char  
*argv[])
```

where $i = 0, \dots, n$ and $+$ means one or more times.

The difference between these 4 system calls has to do with syntax.

`execl()` and `execv()` require the whole pathname of the executable program to be supplied.

`execlp()` and `execvp()` use the variable `$PATH` to find the program.

Ignore the yellow ones for this course

```
#include <unistd.h>

int execl(const char *pathname, const char *arg0, ... /* (char *)0 */ );
int execv(const char *pathname, char *const argv[]);
int execle(const char *pathname, const char *arg0, ...
           /* (char *)0, char *const envp[] */ );
int execve(const char *pathname, char *const argv[], char *const envp[]);
int execlp(const char *filename, const char *arg0, ... /* (char *)0 */ );
int execvp(const char *filename, char *const argv[]);
int fexecve(int fd, char *const argv[], char *const envp[]);
```

All seven return: -1 on error, no return on success

man exec

A successful call to `exec()` never returns.

`exec()` returns -1 if not successful.

For both `execl()` and `execlp()` `arg0` must be the name of the program.

For both `execv()` and `execvp()` `arg[0]` must be the name of the program.

Example using `execl` (`execl1.c`)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("Before: process id %d\n", getpid());

    if ((pid = fork()) == 0) {
        printf("I am the child %d\n", getpid());
        sleep(5);
        printf("Listing content of current directory...\n");
        execl("/bin/ls", "ls", "-l", "-t", (char *) 0);
    }
    else {
        printf("I am the parent %d\n", getpid());
        int status;
        int term_pid = wait(&status);
        printf("Child %d has listed the content of current directory\n",
            term_pid); exit(1);
    }
}
```

Example using `exec1p` (`exec1p1.c`)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("Before: process id %d\n", getpid());

    if ((pid = fork()) == 0) {
        printf("I am the child %d\n", getpid());
        sleep(5);
        printf("Listing content of current directory...\n");
        exec1p("ls", "ls", "-l", (char *)0);
    }
    else {
        printf("I am the parent %d\n", getpid());
        int status;
        int term_pid = wait(&status);
        printf("Child %d has listed the content of current directory\n", term_pid);
        exit(1);
    }
}
```

Example using `execv` (`execv1.c`)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("Before: process id %d\n", getpid());

    if ((pid = fork())==0){
        printf("I am the child %d\n", getpid());
        sleep(5);
        printf("Listing content of current directory...\n");
        char* arg_list[3] = {"ls", "-l", (char *)0};
        // OR
        // char* arg_list[3];
        // arg_list[0] = "ls";
        // arg_list[1] = "-l";
        // arg_list[2] = 0;

        execv("/bin/ls", arg_list);
    }
    else{
        printf("I am the parent %d\n", getpid());
        int status;
        int term_pid = wait(&status);
        printf("Child %d has listed the content of current directory\n", term_pid);
        exit(1);
    }
}
```

Example using `execvp` (`execvp1.c`)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("Before: process id %d\n", getpid());

    if ((pid = fork())==0){
        printf("I am the child %d\n", getpid());
        sleep(5);
        printf("Listing content of current directory...\n");
        char* arg_list[3] = {"ls", "-l", (char *)0};
        // OR
        // char* arg_list[3];
        // arg_list[0] = "ls";
        // arg_list[1] = "-l";
        // arg_list[2] = 0;

        execvp("ls", arg_list);
    }
    else{
        printf("I am the parent %d\n", getpid());
        int status;
        int term_pid = wait(&status);
        printf("Child %d has listed the content of current directory\n", term_pid);
        exit(1);
    }
}
```

Part II

Changing Directory: `chdir()`



Inheriting and changing directory

A child process inherits its current working directory from its parent.

Each process can change its working directory using `chdir()`.

`chdir()` synopsis:

```
int chdir(const char * pathName);
```

`chdir()` returns 0 if successful -1 otherwise.

It fails if the specified path name does not exist or if the process does not have execute permission from the directory.

Part III

Scheduling priority



System scheduling priority

Every process has a system scheduling priority.

Each process runs at a **default system priority**: the priority is inherited from its parent.

The numerical range of priority values is: -20 to 19. Only the super-user can get negative values. However, this range differs from one Unix platform to another.

The scheduling priority can be changed using the `nice()` system call.

Priority value can also be changed using `setpriority()`.

Priority value can be obtained using `getpriority()`.

The priority value affects the amount of CPU time allocated to the process. **The higher the value, the lower the process' priority.**

man 2 nice

NOT REQUIRED FOR MIDTERM pg 11 - 25

Changing priority `nice()`

Synopsis:

```
int nice(int delta);
```

`nice()` adds `delta` to the process current priority value. Note that only super-user processes can have a negative priority value.

`nice()` returns the new priority value if successful and -1 otherwise.

`getpriority()` and `setpriority()`

`getpriority()`

Synopsis: `int getpriority(int which, id_t who);`
obtains the current scheduling priority of a process, process group, or user. Returns the priority value on success, -1 on failure.

`setpriority()`

Synopsis: `int setpriority(int which, id_t who, int priority);`
sets the scheduling priority of a process, process group, or user. Returns 0 on success, -1 on failure.

The two functions require

`#include <sys/resource.h>`

`which` identifies whether it is a process (`PRIO_PROCESS`), process group...etc.

`who` interpreted as a process id, group id,...etc. If `who` is 0, the calling process, (or group,...etc) is considered.

`priority` is the priority value (in the appropriate range).

Example... (priority.c)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("I am the parent %d. My priority is %d\n",
        getpid(), getpriority(PRIO_PROCESS, getpid()));

    if ((pid = fork()) == 0) {
        printf("I am the child %d. My priority is %d\n",
            getpid(), getpriority(PRIO_PROCESS, getpid()));
        printf("Child: nice(19) returns %d\n", nice(19));
        printf("I am the child %d. My priority is now %d\n",
            getpid(), getpriority(PRIO_PROCESS, getpid()));
        exit(20);
    }
    else {
        sleep(5);
        printf("Changing parent's priority...\n");
        printf("Parent: nice(10) returns %d\n", nice(10));
        int status;
        int term_pid = wait(&status);

        int new_pid = fork();
        if (new_pid == 0) {
            printf("I am the new child %d. My priority is %d\n",
                getpid(), getpriority(PRIO_PROCESS, getpid()));
            printf("New child: nice(5) returns %d\n", nice(5));
            printf("I am the new child %d. My priority is now %d\n",
                getpid(), getpriority(PRIO_PROCESS, getpid()));
            exit(21);
        }
        else term_pid = wait(&status);
    }
}
```

...

The output of the previous program is:

```
I am the parent 9306. My priority is
0 I am the child 9307. My priority
is 0 Child: nice(19) returns 19
I am the child 9307. My priority is now
19 Changing parent's priority...
Parent: nice(10) returns 10
I am the new child 9308. My priority is
10 New child: nice(5) returns 15
I am the new child 9308. My priority is now
15
```

Example (`priority2.c`)...

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int pid;
    printf("I am the parent %d. My priority is %d\n",
        getpid(), getpriority(PRIO_PROCESS, getpid()));

    if ((pid = fork())==0){
        printf("I am the child %d. My priority is %d\n",
            getpid(), getpriority(PRIO_PROCESS, getpid()));
        setpriority(PRIO_PROCESS, getpid(), 19);
        printf("I am the child %d. My priority is now %d\n",
            getpid(), getpriority(PRIO_PROCESS, getpid()));
        exit(20);
    }
    else{
        sleep(5);
        printf("Changing parent's priority...\n");
        printf("Parent: nice(10) returns %d\n", nice(10));
        int status;
        int term_pid = wait(&status);
    }
}
```

...

The output of the previous program is:

```
I am the parent 13557. My priority is 0
```

```
I am the child 13558. My priority is 0
```

```
I am the child 13558. My priority is now 19
```

```
Changing parent's priority...
```

```
Parent: nice(10) returns 10
```

`nice()` VS `setpriority()`

Function	What It Does	Typical Use
<code>nice(int inc)</code>	Increases (adds) the niceness of the current process by <code>inc</code>	Simple adjustments (relative change)
<code>setpriority(int which, int who, int prio)</code>	Directly sets the niceness to <code>prio</code> for a process, process group, or user	Exact control (absolute change)

Part IV

Process Groups



Process groups (Read Chapter 9, not required for this course)

1. Every process is a member of a process group.
A child inherits its process group from its parent.
2. When a process execs, its process group remains the same.
3. A process may change its process group to a new value: either to become a member of another group or to create its own group (in which the process will initially be the leader and sole member).
4. One of the group members is the group leader.
Each member of the group has the group leader's process-ID as its process-group-ID.

The kernel provides a system call to send a signal to each member of a designated process group.
This would be used to terminate the entire group as a whole, but any signal can be broadcast in this way.

`setpgid()`

```
#include <sys/types.h>  
#include <unistd.h>
```

`setpgid()`

Synopsis: `int setpgid(pid_t pid, pid_t pgid):`

1. sets the process group ID of the process, whose ID is `pid`, to `pgid`. It returns 0 if successful and -1 otherwise.
2. If `pgid` is equal to `pid`, the process becomes a process group leader.
3. If `pid` is equal to 0, the calling process group ID is set to
4. `pgid`.
5. if `pgid` is equal 0, the process specified by `pid` becomes a process group leader.

`getpgid()`

```
#include <sys/types.h>
#include <unistd.h>
```

`getpgid()`

Synopsis: `pid_t getpgid(pid_t pid)`

1. Returns the process group ID of the process with PID equal to `pid`.
2. If `pid` is 0, the calling process group ID is returned.

Example (`pgid1.c`)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/wait.h>

int main(int argc, char *argv[]){

    int pid = getpid();
    printf("Parent: PID = %d, PPID = %d, PGID = %d\n",pid,getppid(),getpgid(pid));

    if ((pid = fork())==0){

        printf("Child: PID = %d, PPID = %d, PGID = %d\n",
            getpid(),getppid(),getpgid(getpid()));
        exit(1);
    }
    sleep(5);
}
```

Output:

Parent: PID = 20814, PPID = 20381, PGID = 20814

Child: PID = 20815, PPID = 20814, PGID = 20814

Another example (`pgid2.c`)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/wait.h>

int main(int argc, char *argv[]){

    int pid = getpid();
    printf("Parent: PID = %d, PPID = %d, PGID = %d\n",pid,getppid(),getpgid(pid));

    if ((pid = fork())==0){

        printf("Child: PID = %d, PPID = %d, PGID = %d\n",getpid(),getppid(),getpgid(getpid()));

        setpgid(0,0); //or setpgid(getpid(),0);
        printf("Child after setpgid: PID = %d, PPID = %d, PGID = %d\n",
            getpid(),getppid(),getpgid(getpid()));

    }
    sleep(5);
}
```

Output:

Parent: PID = 22295, PPID = 20381, PGID = 22295

Child: PID = 22296, PPID = 22295, PGID = 22295

Child after setpgid: PID = 22296, PPID = 22295, PGID = 22296

Part V

Example



A small shell program...

Example

Write a C program, that behaves like a small shell, to process commands entered by the user.

The program should assemble commands and execute them, either in the background or foreground.

The command's / program's location can be anywhere in `$PATH` and might have arguments.

Algorithm

Algorithm:

```
While(1)
begin
  get command-line from user
  assemble command args
  duplicate current process
  child should exec to the new program  if
  (foreground execution) then
    parent process waits for its child  to
    terminate
end
```


More example code for process control

```
fork_example.c  
Exec-example.c  
Execv-example.c  
Myecho.c  
runls3.c  
Docommand.c  
  
Smallsh.h  
Proc_line.c (b 18, b41)  
Userin.c  
Runcommand.c  
Main_smallsh.c (b 11, check inpbuf tokbuf)
```

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGLIPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111111100
00000111011011
11001100111100

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor